

Lab 2: Stellar Data and Surface Gravity

Henry Jugan
ASTRON 128: Spring 2026

April 14, 2026

Abstract

This report documents the process of modeling stellar spectra. The Lab began with querying the SDSS APOGEE catalog. I constructed my query to eliminate dwarf stars from my data set. The key labels were surface gravity ($\log(g)$), effective temperature (T_{eff}), metallicity ($[Fe/H]$), and alpha abundances ($[Mg/Fe]$, $[Si/Fe]$). Next, I utilized bit masking to remove low quality specimen. These spectra were used to create a normalized flux plot, highlighting the star's observed absorption lines. Finally, the data set was used to train an MCMC model that predicted appropriate stellar labels and absorption lines for giant stars.

1 Introduction

This lab focuses on training an MCMC model to predict stellar properties based on five physical parameters; surface gravity ($\log g$), effective temperature (T_{eff}), metallicity ($[Fe/H]$), and alpha abundances ($[Mg/Fe]$, $[Si/Fe]$).

2 Methods

2.1 Data Retrieval

Stellar data was retrieved from the SDSS catalog using the `astroquery` package. The query targeted four specific fields and filtered out missing data, low signal-to-noise ratio, dwarf stars, and low metallicity.

```
1 from astroquery.sdss import SDSS
2
3 # SQL query to retrieve stellar data
4 # Select identifiers, field location, physical labels, and SNR in the allStar
   catalog
5 # Apply conditions to filter out missing data, low SNR, dwarf stars, and low
   metallicity
6 # I used the IN operator to limit the results to the four specified fields
7 # The != -9999 argument filters out the APOGEE filler value to ensure only valid
   labels are kept
8 # I had to separately query %060+00 since the % sign was messing up the query,
   making me unable to access the data set!
9 query = """
10 SELECT TOP 3000
```

```

11     s.apogee_id, s.field, s.telescope, s.snr, a.teff, a.logg, a.fe_h, a.mg_fe, a.
    si_fe
12 FROM
13     apogeeStar s
14 JOIN
15     aspcapStar a ON s.apstar_id = a.apstar_id
16 WHERE
17     (s.field IN ('M15', 'N6791', 'K2_C4_168-21') OR s.field LIKE '%060+00')
18     AND a.teff != -9999
19     AND a.logg != -9999
20     AND a.fe_h != -9999
21     AND a.mg_fe != -9999
22     AND a.si_fe != -9999
23     AND s.snr >= 50
24     AND a.logg <= 4
25     AND a.teff <= 5700
26     AND a.fe_h >= -1
27 ""
28
29 # Execute the query and convert to a pandas DataFrame.
30 data = SDSS.query_sql(query, data_release=17, cache=False)
31 df = data.to_pandas()
32
33 # Display the number of rows retrieved
34 print(f"stars retrieved: {len(data)}")

```

Listing 1: Retrieving APOGEE Data via SDSS

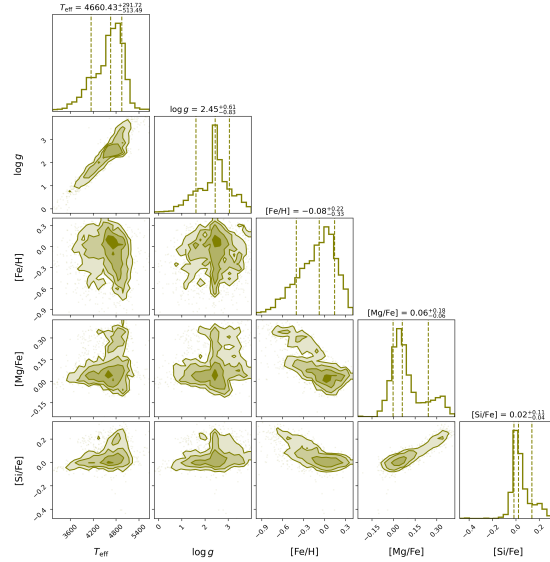


Figure 1: SDSS Queried APOGEE Giant Stars Corner Plot in Label Space

2.2 How Radius Determines Star Type

The code below exemplifies how stellar mass and radius effects stellar gravity. The function calculates the $\log(g)$ of a few types of stars. It shows how the $\log(g) < 4$ cut affected the SDSS query.

```
1 import math
2
3 def calculate_logg(mass_solar, radius_solar):
4     """
5     Calculates the log (base 10) of stellar surface gravity.
6     Accepts mass and radius in solar units and performs calculations in cgs units.
7     """
8     g = 6.6743e-8
9     m_sun = 1.9884e33
10    r_sun = 6.957e10
11    mass_g = mass_solar * m_sun
12    radius_cm = radius_solar * r_sun
13
14    # Calculate surface gravity using the formula g = GM/R^2.
15    surface_gravity = (g * mass_g) / (radius_cm ** 2)
16
17    # Calculate the base-10 logarithm of the surface gravity.
18    logg = math.log10(surface_gravity)
19
20    return logg
21
22 # Determines the log(g) value on the main sequence.
23 main_sequence_logg = calculate_logg(1.0, 1.0)
24
25 # Determines the log(g) value just before the helium flash.
26 helium_flash_logg = calculate_logg(1.0, 100.0)
27
28 # Determines the log(g) value during core helium burning.
29 core_burning_logg = calculate_logg(1.0, 15.0)
30
31 o_type_logg = calculate_logg(25.0, 11.0)
32
33 print(f"Main Sequence log(g): {main_sequence_logg:.2f}")
34 print(f"Helium Flash log(g): {helium_flash_logg:.2f}")
35 print(f"Core Helium Burning log(g): {core_burning_logg:.2f}")
36 print(f"O-type Main sequence log(g): {o_type_logg:.2f}")
```

Listing 2: Calculating Surface Gravity

Calculated Values:

- Main Sequence $\log(g)$: 4.44
- Helium Flash $\log(g)$: 0.44
- Core Helium Burning $\log(g)$: 2.09
- O-type Main sequence $\log(g)$: 3.75

2.3 Bitmask Quality Cuts

APOGEE observes spectra with a multi pixel array. Each pixel has bitwise flags associated with it that indicate quality issues. For my quality cuts I removed pixels that had bits 0-7 and 12 flipped on. This was achieved with a simple function that utilized bitwise arithmetic.

```
1  def clean_spectrum(file_name):
2
3  # Open FITS file
4  with fits.open(file_name) as hdul:
5      flux = hdul[1].data[0] # extract flux array.
6      error = hdul[2].data[0] # Extract error array.
7      bitmask = hdul[3].data[0] # extract the bitmask array.
8
9  bits_0_to_7 = 255 # since im filtering for bits 0-7 i set the bit value to 255
   (8 1's in binary)
10
11 # integer value for bit 12
12 # Uses a bitwise left-shift to calculate 2^12.
13 bit_12 = 1 << 12
14
15
16
17 # Combine the targeted bits using a bitwise OR operator.
18 # Creates a single integer that I can use to filter through the bits
19
20 critical_flags = bits_0_to_7 | bit_12
21
22
23
24 # AND operator to see if a pixel has a corresponding flag
25 bad_pixels_index = (bitmask & critical_flags) != 0
26
27
28
29 # Inflate the uncertainty of the flagged pixels
30 # Assigning a massive error value so the pixels weight will be basically zero
   when we use it
31
32 error[bad_pixels_index] = 1e10
33
34 return flux, error
35
36
37
38 clean_flux, clean_error = clean_spectrum(file_name)
39
40
41 print(f"Total pixels: {len(clean_flux)}")
42
43 print(f"Pixels filtered out: {np.sum(clean_error == 1e10)}")
```

Listing 3: Bitmask quality cuts on APOGEE pixel array

2.4 Pseudo Continuum Spectrum Normalization

The next goal was to identify the absorption lines of the stars. I adopted my process from Ness et al. 2015. with slight tweaks to the polynomial fitting algorithm. I created a pseudo spectrum with the bitmasked pixel array, and then approximated the star’s spectrum with a degree 4 Chebyshev polynomial. In addition, I implemented a sigma clipping algorithm to catch extreme spectral outliers. I removed pixels affected by cosmic rays with a 3 sigma upper bound cut off. Pixels that suffered from deep absorption were weeded out with a .5 sigma lower bound.

2.5 Stellar Model Construction

I built my model by constructing a design matrix. For a fixed value of intrinsic scatter (s_λ^2), the spectral model for a single wavelength pixel is formulated as the following linear equation:

$$X\theta_\lambda = f_\lambda \quad (1)$$

f_λ is an $N_{\text{stars}} \times 1$ column vector containing the normalized flux at wavelength λ . It has a row for every star in the training set. The vector θ_λ represents the free parameters (polynomial coefficients) for that specific wavelength. The term X represents the design matrix. It has dimensions $N_{\text{stars}} \times N_{\text{params}}$. The model applies a second-order polynomial transformation to five stellar labels (T_{eff} , $\log g$, $[\text{Fe}/\text{H}]$, $[\text{Mg}/\text{Fe}]$, and $[\text{Si}/\text{Fe}]$), and X maps the known training set labels into polynomial terms for each star.

The parameter vector θ_λ contains 21 independent coefficients:

- 1 constant term (intercept).
- 5 linear terms (one for each individual label).
- 5 quadratic terms (the square of each label).
- 10 cross-product terms (the interaction between every unique pair of labels, calculated as the combination $\binom{5}{2}$).

The inclusion of the intrinsic scatter term, s_λ^2 , introduces non-linearity into the model. As a result I had to include a maximum likelihood estimation. Assuming Gaussian noise, the variance in the observed normalized flux is defined as $s_\lambda^2 + \sigma_{i,\lambda}^2$. $\sigma_{i,\lambda}$ is the observational uncertainty. The optimal parameters are determined by maximizing the following log-likelihood function:

$$\ln \mathcal{L}(\theta_\lambda, s_\lambda^2) = -\frac{1}{2} \sum_{i=1}^{N_{\text{stars}}} \left[\frac{(f_{i,\lambda} - \vec{x}_i \cdot \theta_\lambda)^2}{\sigma_{i,\lambda}^2 + s_\lambda^2} + \ln(2\pi(\sigma_{i,\lambda}^2 + s_\lambda^2)) \right] \quad (2)$$

In this equation, $f_{i,\lambda}$ is the observed flux for star i at wavelength λ , and $\vec{x}_i$ is the i -th row of the design matrix corresponding to the polynomial label terms for star i . Optimization routines iteratively minimize the negative of this function to simultaneously compute the best-fit θ_λ coefficients and the intrinsic scatter s_λ^2 .

The spectral model is constructed by concatenating the individual models evaluated at every wavelength pixel! The log likelihood function is optimized for each pixel. This means 22 parameters

are calculated, the 21 polynomial coefficients (θ_λ) and the intrinsic scatter term (s_λ^2). For an APOGEE spectral grid containing N_{pixels} valid wavelengths, the total number of free parameters optimized during the training process is $22 \times N_{\text{pixels}}$.

2.6 Mystery Spectrum Evaluation with MCMC Methods

I used Bayesian analysis to extract the posterior distribution of the stellar labels. Uniform prior distributions were adopted for all five stellar labels (T_{eff} , $\log g$, $[\text{Fe}/\text{H}]$, $[\text{Mg}/\text{Fe}]$, and $[\text{Si}/\text{Fe}]$). The prior boundaries were defined by physical limits and the extent of the training set data. Any proposed step falling outside these predetermined bounds was assigned a prior probability of zero. This ensured the chains remained strictly confined to physical regions of the label space.

I sampled the posterior distribution using a custom Metropolis-Hastings algorithm. At each step, a new set of labels was proposed by drawing from a multi-dimensional Gaussian distribution centered on the current state. The proposed state was evaluated by calculating the log-posterior, which is the sum of the log-prior and the log-likelihood. The Gaussian log-likelihood incorporated both the observational uncertainties and the previously determined intrinsic scatter model. A proposed state was accepted if the Metropolis acceptance criterion was met:

$$\ln(u) < \ln(\mathcal{L}_{\text{proposed}}) - \ln(\mathcal{L}_{\text{current}}) \quad (3)$$

where u is a random number drawn from a uniform distribution $\mathcal{U}(0, 1)$.

3 Results

3.1 Surface Gravity Cut

The $\log(g)$ values for various evolutionary stages were computed using solar mass and radius parameters.

```

1 import math
2
3 def calculate_logg(mass_solar, radius_solar):
4     """
5     Calculates the log (base 10) of stellar surface gravity.
6     Accepts mass and radius in solar units and performs calculations in cgs units.
7     """
8     g = 6.6743e-8
9     m_sun = 1.9884e33
10    r_sun = 6.957e10
11    mass_g = mass_solar * m_sun
12    radius_cm = radius_solar * r_sun
13
14    # Calculate surface gravity using the formula g = GM/R^2.
15    surface_gravity = (g * mass_g) / (radius_cm ** 2)
16
17    # Calculate the base-10 logarithm of the surface gravity.
18    logg = math.log10(surface_gravity)
19
20    return logg
21
22 # Determines the log(g) value on the main sequence.
```

```

23 main_sequence_logg = calculate_logg(1.0, 1.0)
24
25 # Determines the log(g) value just before the helium flash.
26 helium_flash_logg = calculate_logg(1.0, 100.0)
27
28 # Determines the log(g) value during core helium burning.
29 core_burning_logg = calculate_logg(1.0, 15.0)
30
31 o_type_logg = calculate_logg(25.0, 11.0)
32
33 print(f"Main Sequence log(g): {main_sequence_logg:.2f}")
34 print(f"Helium Flash log(g): {helium_flash_logg:.2f}")
35 print(f"Core Helium Burning log(g): {core_burning_logg:.2f}")
36 print(f"O-type Main sequence log(g): {o_type_logg:.2f}")

```

Listing 4: Calculating Surface Gravity

Calculated Values:

- Main Sequence $\log(g)$: 4.44
- Helium Flash $\log(g)$: 0.44
- Core Helium Burning $\log(g)$: 2.09
- O-type Main sequence $\log(g)$: 3.75

3.2 Normalized Spectrum

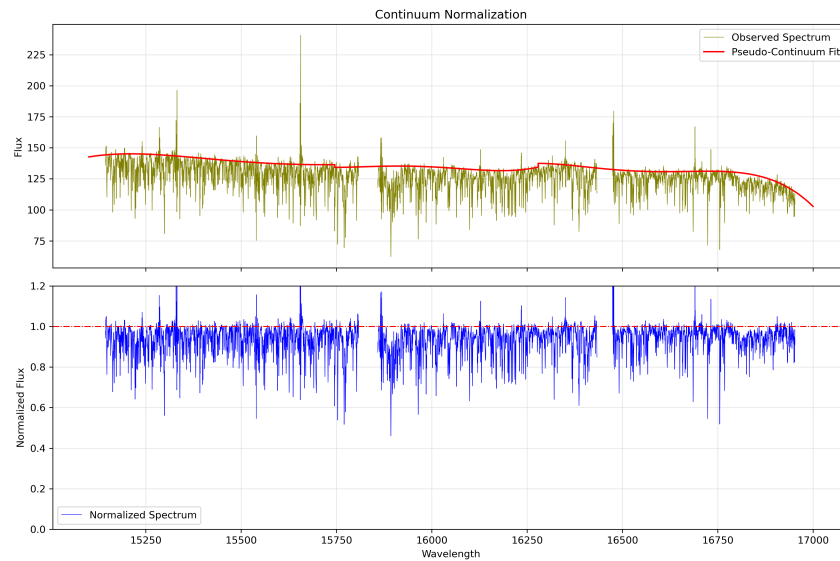


Figure 2: Upper: Pseudo Spectrum Generated with a Chebyshev Polynomial and Sigma Clipping Algorithm
Lower: Normalized Spectrum Highlighting Deep Absorption Lines

3.3 Stellar Model Verification

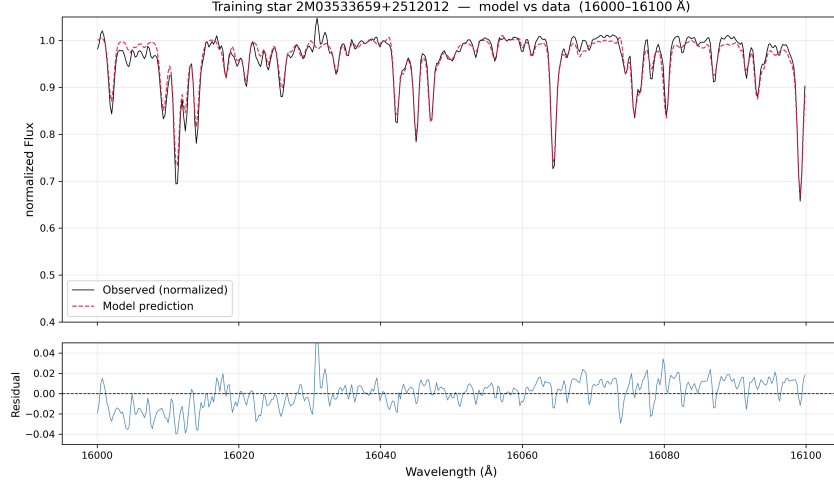


Figure 3: 2M03533659+2512012 Model vs. Observed Spectrum for Wavelength Range 16000Å - 16100Å

In order to verify that my model was working properly I over plotted the normalized spectrum of star 2M03533659+2512012 and the model spectrum predicted for its labels.

3.4 Cross Validation Check

The cross-validation results reveal subsets of stars that deviate substantially from the ASPCAP values. These outliers suggest that the non-linear optimizer occasionally settled into a local minimum. The 5-dimensional χ^2 surface contains multiple minima. It's possible that the initial parameter guess that I gave to the optimizer is too far from the global minimum. If so, this could have caused the algorithm to converge on an incorrect set of stellar labels.

The source of this issue is likely attributed to my pseudo continuum. The polynomial fit seems inaccurate. It does not resemble an idealized blackbody curve. It doesn't seem far fetched to infer that this is due to excessive observational noise. This could be fixed with additional bitmask flags, and quality cuts to individual spectra.

The ASPCAP pipeline relies on specific bitmasks (e.g., `ASPCAPFLAG` and `STARFLAG`) to indicate potential issues with the derived labels. It's possible that several of the identified outliers possess crucial flags that I did not include in my bitmask. This means the reference labels used for comparison may be inherently unreliable.

Modifications that could improve my model's robustness include:

- A routine that initializes the optimizer at multiple different points in the label space. This would select the result with the lowest overall χ^2 to mitigate the local minimum problem.
- Apply stricter bitmasking and sigma-clipping during the pseudo-continuum normalization phase. I suspect my data set was rampant with artifact-driven distortions.

- Filter the cross-validation set by ASPCAP quality flags to ensure the model is evaluated exclusively against high-confidence data points.

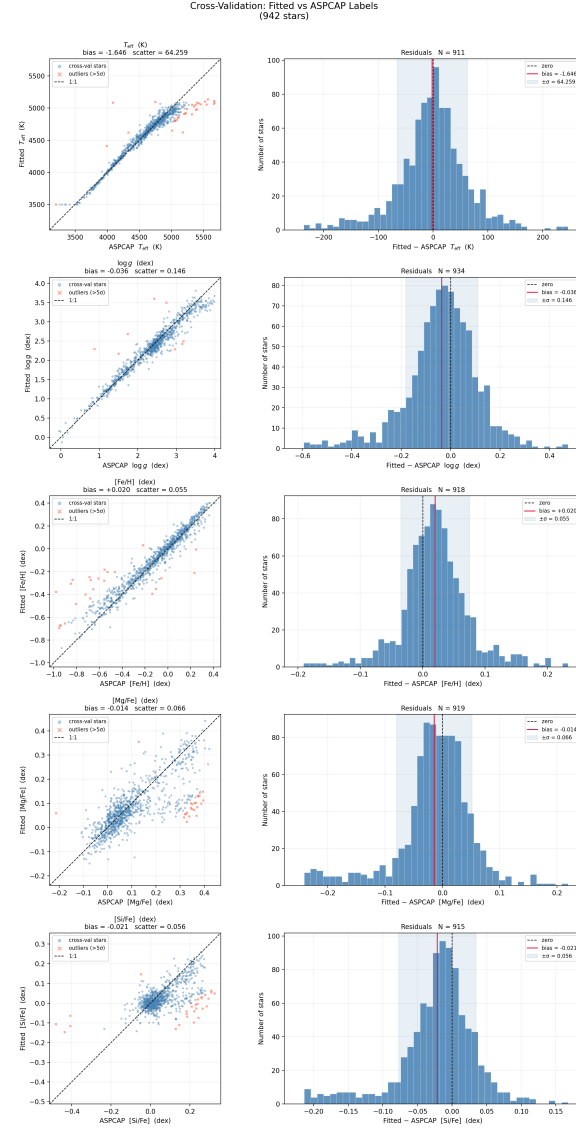


Figure 4: Model Diagnostics when Applied to Cross Validation Set

3.5 Kiel Diagram & Isochrone Fit

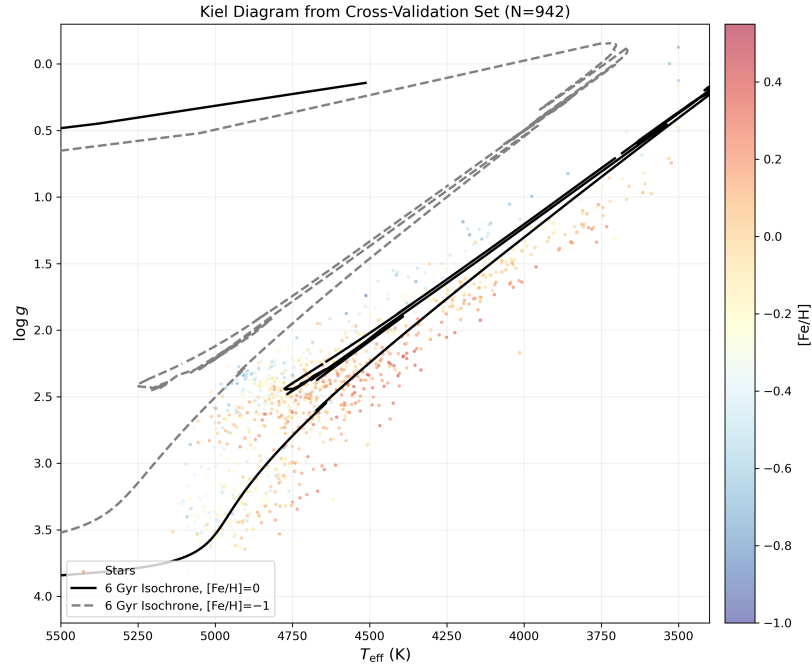


Figure 5: Kiel Diagram Generated from Cross Validation Set with Overplotted MIST Isochrones

See Section 4.6 for an analysis of these results.

3.6 MCMC Results

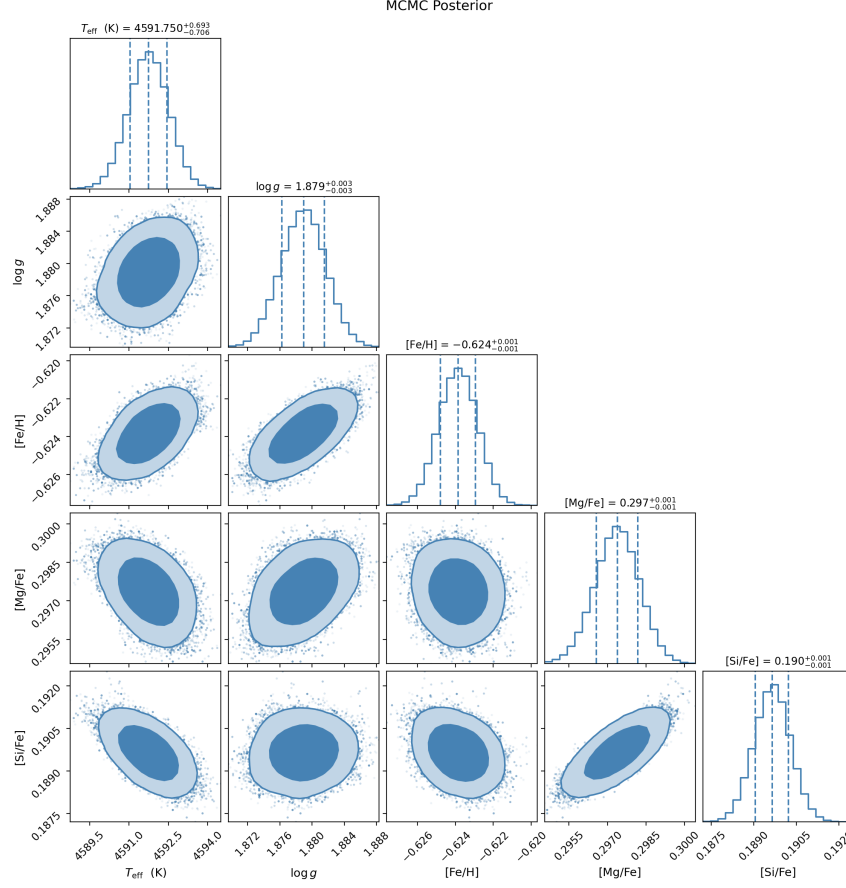


Figure 6: MCMC Generated Labels from Mystery Spectrum

The corner plot is theoretically consistent. It shows correlations in areas that it should. For example, surface gravity and metallicity have a strong positive correlation, which is expected because a star with more heavy elements has a greater mass. Despite this, the error bars raise a red flag. They are impossibly low, which hints at the fact that my model may not have explored the entire distribution.

Typical uncertainties reported by the ASPCAP pipeline are roughly 30 K for T_{eff} and 0.02 to 0.05 dex for elemental abundances. Also, the scatter derived from the cross-validation set is significantly larger than the MCMC formal errors. The MCMC results do not represent the accuracy of the model.

It's also possible that the extreme underestimation is due to the log-likelihood function. The likelihood evaluates the χ^2 across thousands of wavelength pixels, treating each pixel as an independent data point. This drives the standard error down proportional to $1/\sqrt{N_{\text{pixels}}}$.

In contrast, my trace plots help instill confidence in my model. The erratic and fuzzy graphs indicate that the model spent time in many different areas of the distribution, bouncing around from one area to the next. These are hallmarks of MCMC convergence.

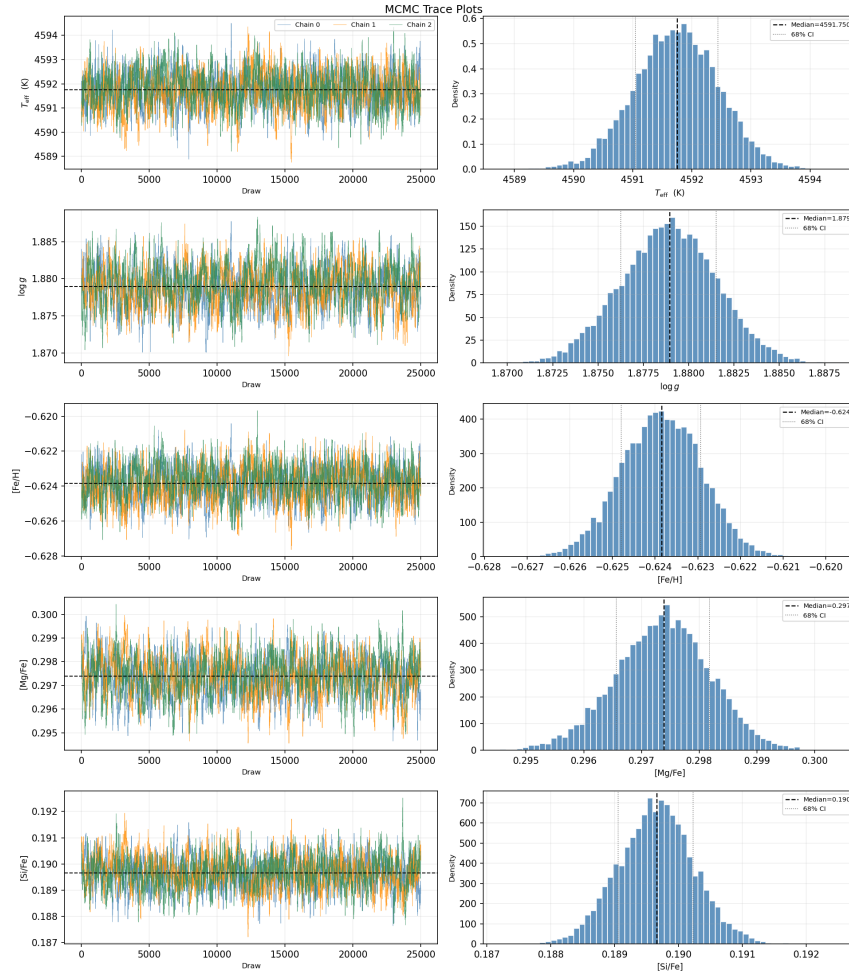


Figure 7: Mystery Spectrum MCMC Trace Plots

A posterior predictive check shows that my model did in fact explore the distribution. This allows me to infer that the error bound issue is a result of the log likelihood function.

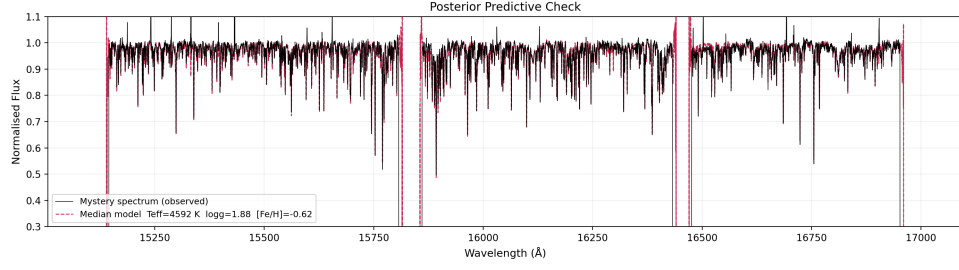


Figure 8: Posterior Predictive Check of Mystery Spectrum. A quick glance suggests model convergence.

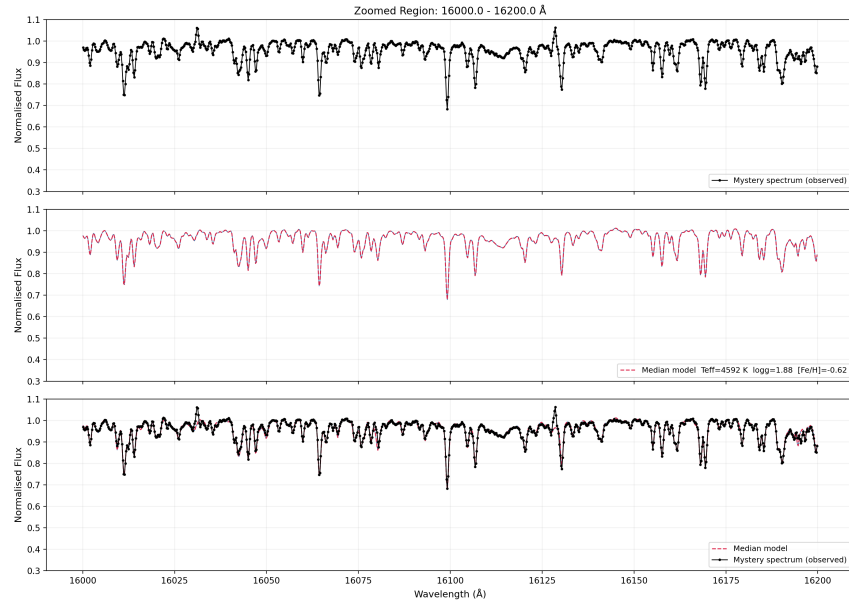


Figure 9: A zoomed in look at the Posterior Predictive Check. This helps distinguish between MCMC noise and genuine model exploration. The model converged to absorption lines, and moved throughout the distribution. This result allows me to conclude that the log-likelihood function caused my uncertainty errors.

3.7 Identifying Stars with MCMC Labels

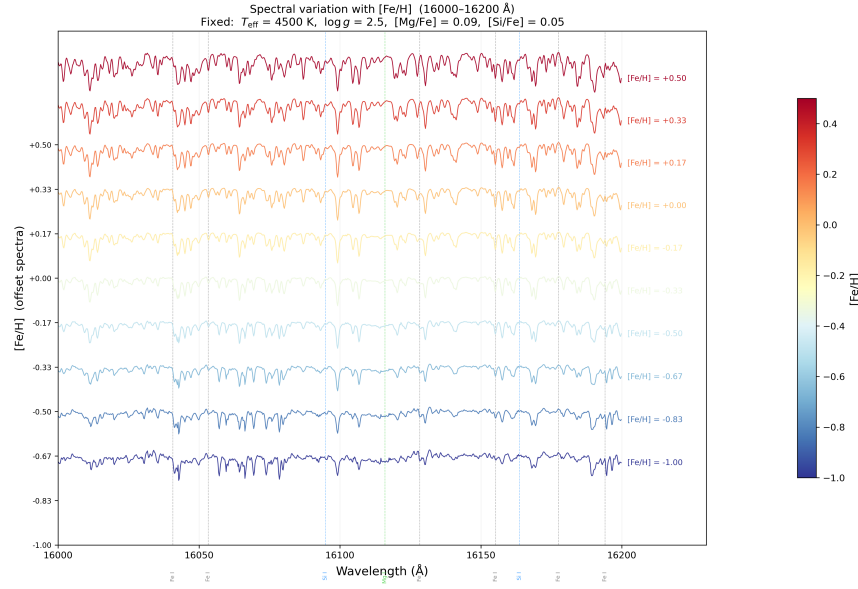


Figure 10: Spectral Variance of different Metallicities. $[Fe/H]$ as a function of Wavelength; Sequenced for $-1.0 \leq [Fe/H] \leq 0.5$

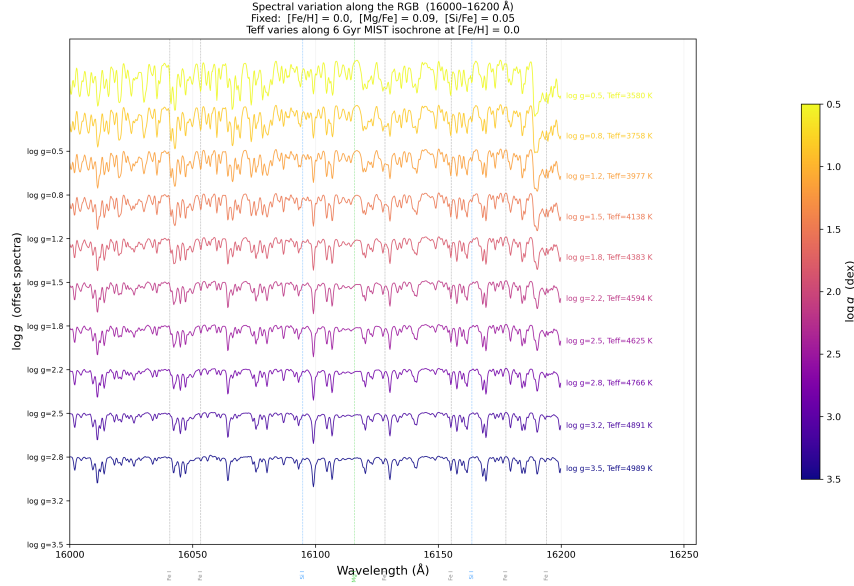


Figure 11: A plot using color and offset spectra to show how the same region of the spectrum changes as a star ascends the red giant branch.

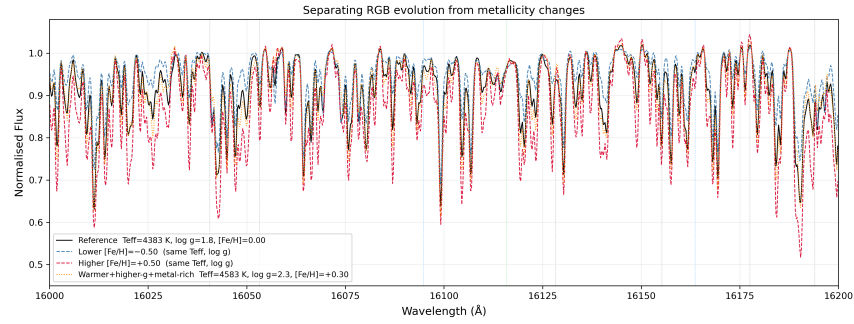


Figure 12: RGB vs. Metallicity Comparison

These plots help allow us to distinguish between cool stars with low surface gravity, and warm stars with high surface gravity and high metallicity.

The resulting line depths appear similar, but the degeneracy can be broken by analyzing the ratios of specific spectral features. The comparison plot separating RGB evolution from metallicity changes illustrates that the “warmer + higher- g + metal-rich” spectrum does not perfectly overlap the reference spectrum across all wavelengths.

Models distinguish between these scenarios by relying on lines with differing physical sensitiv-

ities. Lines that come from lower energy levels respond differently to temperature changes. This means that the ratio of a high-excitation line to a low-excitation can be used to parse the star's temperature without having to consider its metallicity. Additionally, the ratio of absorption from neutral species to singly ionized species (e.g., Fe I vs. Fe II) is highly sensitive to electron pressure. Electron pressure is a phenomenon that arises because of the star's gravity, and it increases proportionally. Also, molecular features are sensitive to temperature drops. These absorption lines also help to tell a cool star from a metal-rich warm star.

4 Discussion

4.1 Surface Gravity Cut

It is important to note that all "dwarf stars" are also main sequence stars. The $\log(g)$ cut removes dwarf stars from the data set. Surface gravity is proportional to stellar mass and the star's radius squared (GM/R^2). Dwarf stars have small radii relative to their mass. This results in high values of surface gravity. Giant stars have radii that are disproportionately large compared to their mass. This causes their measured surface gravity to plummet. A priori, a $\log(g) < 4$ cut removes stars with high surface gravity, i.e. dwarf stars.

4.2 Doppler Shifts and the Barycentric Reference Frame

Earth's orbit around the Sun causes inconsistencies in stellar spectra measurements. Earth moves 30km/s around the sun; inducing a Doppler shift unto observations (blue shifting while moving toward the star, red shifting while moving away). This shifted spectra can be fixed by subtracting Earth's motion from observations. This transforms the reference frame to the barycenter of the solar system, effectively removing Doppler shift noise that originates from Earth's orbit.

4.3 Flux vs. Wavelength

Wavelength is a property of the light that is being measured. It is unique to the color of the light, and the energy that it possesses.

Flux is a measurement of how much light is produced by a source, per second, through an area, at a specific wavelength. It's a mouthful but its a very useful quantity to measure. Since we know how much energy is intrinsic to a specific wavelength of light, we can measure the energy output of the star!

The units of flux are ergs per second ($\frac{erg}{s}$) per cm^2 per Angstrom ($\text{\AA}$). What this means is that flux is a measurement of the density of energy output. The units of flux say, "At a specific wavelength ($\text{\AA}$), this much energy per second ($\frac{erg}{s}$) is being passed through a surface (cm^2)".

4.4 Psuedo Continuum vs. "True" Continuum Normalization

A continuum we are refers to a star's Flux vs Wavelength plot. A graph showcasing its energy output at any particular wavelength. A star's "true continuum" is characterized by its blackbody curve. It's the curve you would expect to see in a theoretical setting where the star is a perfect

blackbody. This is not even close to what we see when we observe stars. Especially giant stars which have complex atmospheres. Elements in the atmosphere absorb a lot of the starlight, and re-emit it at lower wavelengths. This dampens the observed flux at particular wavelengths giving us inaccurate measurements.

In order to reconcile with this, we construct a "Pseudo continuum" to approximate the star's theoretical lightcurve. The process is as follows: certain wavelengths have very low absorption and are almost true to what we'd expect to observe. We use these data points to generate a polynomial fit of the lightcurve. This approximate lightcurve is used to normalize the flux by dividing the raw flux data by the pseudo continuum. Places with good data (the training points) should become equal to 1 since you're essentially dividing the data point by itself. Wavelengths with high absorption will dip below 1. Since each element has unique absorption spectra, the normalized spectrum creates a guidebook that can be used to understand the composition of the star's atmosphere!

4.5 Gradient Spectrum and Intrinsic Scatter Plots

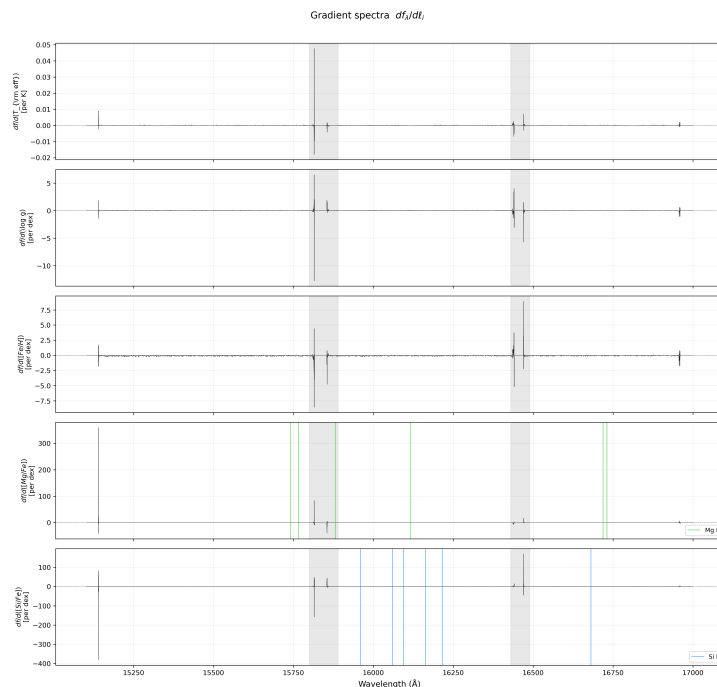


Figure 13: The Gradient Spectrum df_{λ}/dl_i for each Stellar Label

My plots show no correlation between the gradient spectrum, intrinsic scatter, and Mg/Si absorption lines. This result raises a red flag. If a wavelength contains a strong Silicon absorption line there will be less flux at that wavelength. As a result the normalized spectrum will have a sharp change, which would be reflected in a plot of the gradient spectrum. This could be explained by the star that I sampled. If its atmosphere has an atmosphere that is devoid of alpha elements, the

absorption features from those elements wouldn't be present in a plot. Another explanation is that my pseudo continuum spectrum was inaccurate, creating poor absorption lines.

The intrinsic scatter vs. alpha absorption lines plot should not have any correlation. Intrinsic scatter is a metric that scales with variance that my model does not account for. It would be clumped up around absorption lines that are not related to my labels. For example, an absorption feature in a spectrum due to carbon would generate a lot of intrinsic scatter, because my model is only searching for Silicon and Magnesium lines.

4.6 Implications of the Isochrone Fit to the Kiel Diagram

The plot shows a relationship between the $[Fe/H]$ of the Giants and T_{eff} . Heavy element abundance is inversely proportional to T_{eff} . This makes intuitive sense. Heavy elements cause starlight to be absorbed and re-emitted at lower wavelengths. This loss of energy results in a cooler photosphere. The cross validation agrees with the 0.0 $[Fe/H]$ isochrone. I suspect a sampling bias is responsible for this exceptional fit. When constructing the SDSS query, stars with $[Fe/H] < 1$ were filtered out of the data set. This would skew the metallicity to higher values. In addition to this, it is reasonable to assume that the stars within the clusters I queried hover toward similar metallicities. This is because the stars form at similar times. As it turns out, N6791 is a relatively young cluster. The stars within it have high metallic abundances, $[Fe/H] \simeq 0.4$, whereas M61 is a very old cluster, $[Fe/H] \simeq -1.3$. Many of M61's stars were filtered out with my SDSS request, shifting my data set's mean metallicity to a solar abundance.

4.7 Noise from Binary Systems

Binary systems with small angular separation artificially inflate the observed flux of any star within the system. This happens because flux is an additive quantity. What you are really seeing is the flux from two stars that appear to originate from the same point. This can affect the label generation and association of the model. For example, a greater flux implies a greater T_{eff} . This is an issue because this now implies that the star is of lower metallicity, changing the expected magnitude of the absorption lines. This throws off the models label generation, associating greater Brightness temperatures with false absorption lines.

A fix for this would be to filter out stars that have a binary system flag in APOGEE. I could also add a check that compares the theoretically expected flux from stars and their corresponding metallicities to the ones in my data set.

5 Conclusion

The purpose of this lab was to implement a data-driven generative model to predict core stellar properties (T_{eff} , $\log g$, $[Fe/H]$, $[Mg/Fe]$, and $[Si/Fe]$). The model was trained using the APOGEE data set. I constructed a second-order polynomial expansion and optimized it using maximum likelihood estimation. The model used this to map specific stellar labels to normalized observed fluxes.

Cross-validation against ASPCAP reference labels confirmed the model's predictive accuracy. However, severe outliers highlighted the optimization routine's shortcomings. I learned that it is

weak when confronted with local minima and normalization artifacts.

For the rest of the lab I wrote and used a Metropolis-Hastings Markov Chain Monte Carlo algorithm. The model provided me with optimized label estimates for a mystery spectrum. An analysis of the MCMC posterior distributions revealed that the statistical uncertainties were severely underestimated. I chalked up this discrepancy to the likelihood function's assumption of statistically independent wavelength pixels. Such a mathematical model fails to account for the physical and instrumental correlations present in adjacent spectral data points.

This Lab helped me understand the uses and limitations of machine learning models.